ICR_PowerAnalysis

Adam Kiehl

2025-10-15

```
# Set simulation size per N
reps <- 1000
# Maximum sample size
N_min <- 10
N_max <- 600
steps <- 10
```

Question 1: Between-Coder Group ICR

Does inter-coder reliability vary significantly between coder groups (student, clinician, coder)?

$$Y_{r,g} = \beta_0 + \beta_{\text{AI}} \cdot \mathbf{1}\{\text{Cond}_r = \text{AI}\} + \beta_{\text{Student}} \cdot \mathbf{1}\{g = \text{Student}\} + \beta_{\text{Clinician}} \cdot \mathbf{1}\{g = \text{Clinician}\} \tag{1}$$

$$+ \beta_{\text{AI}\times\text{Student}} \cdot \mathbf{1}\{\text{Cond}_r = \text{AI}\} \cdot \mathbf{1}\{g = \text{Student}\} + \beta_{\text{AI}\times\text{Clinician}} \cdot \mathbf{1}\{\text{Cond}_r = \text{AI}\} \cdot \mathbf{1}\{g = \text{Clinician}\} \tag{2}$$

$$+ u_r + \varepsilon_{r,g}, \tag{3}$$

$$u_r \sim \mathcal{N}(0, \sigma^2_{\text{record}}) \tag{4}$$

$$\varepsilon_{r,g} \sim \mathcal{N}(0, \sigma^2). \tag{5}$$

```
# Q1 POWER: Group effect on ICR (mean pairwise Jaccard per record×group)
# Design: 3 groups, 2 coders/group (=> one Jaccard per record×group), N records, 50% records AI (record
# Model: logit(ICR) ~ Group + Condition + Group×Condition + (1|Record)
# Test: Group main effect (Type III). Alpha = 0.05. Power across N = 10..300 by 10.
# ------------------------------------------------------------------------
# This script estimates statistical power (probability of detecting a true effect)
# for the hypothesis "Group main effect does not equal 0" in a mixed-effects model where the outcome
# is the logit-transformed inter-coder reliability (ICR; Jaccard index).
# We simulate many synthetic datasets for each N, fit the model, record the p-value
# for the group main effect, and compute power as the proportion of p-values < alpha.
# ------------------------------------------------------------------------

# Set random seed
set.seed(42)  # Ensures reproducibility of the random simulation results across runs

# ---- ASSUMPTIONS TO SET (PLACEHOLDERS; REVISE AFTER LIT REVIEW) ----
alpha <- 0.05  # Target significance level for hypothesis testing
N_grid <- seq(N_min, N_max, by = steps)  # Range of total records to evaluate power over

# Baseline ICR means (Manual) by group on the probability scale (Jaccard in [0,1])
# Replace with your expectations (e.g., Coders > Clinicians > Students).
mu_manual <- c(Student   = 0.58,  # Expected mean ICR for Students under Manual condition
```

```r
               Clinician = 0.60,  # Expected mean ICR for Clinicians under Manual condition
               Coder     = 0.62)  # Expected mean ICR for Coders under Manual condition
# These encode the size of the "group difference" signal you want to detect.

# AI lift on ICR (absolute difference on probability scale), applied equally across groups for Q1 power
# Set to 0 here, because Q1 focuses on *group* differences, not AI.
delta_AI <- 0.00  # No AI effect when assessing power for the group main effect

# Record random intercept SD on *logit* scale (captures across-record difficulty shared by groups)
sd_record <- 0.50   # PLACEHOLDER; adjust after we review pilot/literature
# Larger sd_record => greater between-record variability in ICR across all groups,
# which generally makes detecting fixed effects harder.

# Beta concentration (phi). Larger => less dispersion around the mean.
phi <- 1.5           # PLACEHOLDER; ensures few 0/1s; adjust as needed
# We simulate ICR values using a Beta distribution around the mean ICR for a record×group.
# phi is the precision parameter: higher phi => tighter (less variable) ICR around the mean.

# ---- Helper functions ----
logit  <- function(p) qlogis(p)  # Convenience alias for logit transform: log(p/(1-p))
invlog <- function(x) plogis(x)  # Inverse-logit: maps real numbers back to (0,1)
clip01 <- function(x, eps = 1e-6) pmin(pmax(x, eps), 1 - eps)  # Avoid exact 0 or 1 (numerical stabilit

simulate_once <- function(N) {
  # Randomly assign half the records to AI
  cond <- sample(rep(c("AI", "Manual"), each = N / 2), size = N, replace = FALSE)
  # Ensure near-balance
  if (abs(sum(cond=="AI") - N/2) > 2) {
    # mild re-balance
    cond[1:floor(N/2)] <- "AI"; cond[(floor(N/2)+1):N] <- "Manual"
    cond <- sample(cond)
  }
  # ^ The block above assigns each of the N records to a condition (AI or Manual),
  # trying to keep them roughly 50/50. Condition is at the RECORD level (all groups share it).

  # Record random intercept (shared across groups)
  u_rec <- rnorm(N, 0, sd_record)
  # ^ u_rec captures record-specific difficulty on the LOGIT scale. It is the same for all groups
  # within a record, inducing correlation among the three group rows for each record.

  # For each record×group, generate ICR via Beta with mean shifted by u_rec on logit scale
  out <- expand.grid(record = seq_len(N),
                     group  = c("Student","Clinician","Coder"),
                     KEEP.OUT.ATTRS = FALSE) |>
    dplyr::mutate(condition = cond[record]) |>
    rowwise() |>
    mutate(
      p_manual = mu_manual[group],                    # Baseline manual mean for this group (probabilit
      p_mean   = ifelse(condition=="AI", p_manual + delta_AI, p_manual),  # Add AI lift (0 for Q1)
      p_mean   = clip01(p_mean),                      # Keep within (0,1) bounds
      eta      = logit(p_mean) + u_rec[record],       # Shifted mean on LOGIT scale by record difficult
      p_star   = invlog(eta),                         # Back to probability scale after adding u_rec
      a        = p_star * phi,                        # Beta shape parameter alpha = mu·phi
```

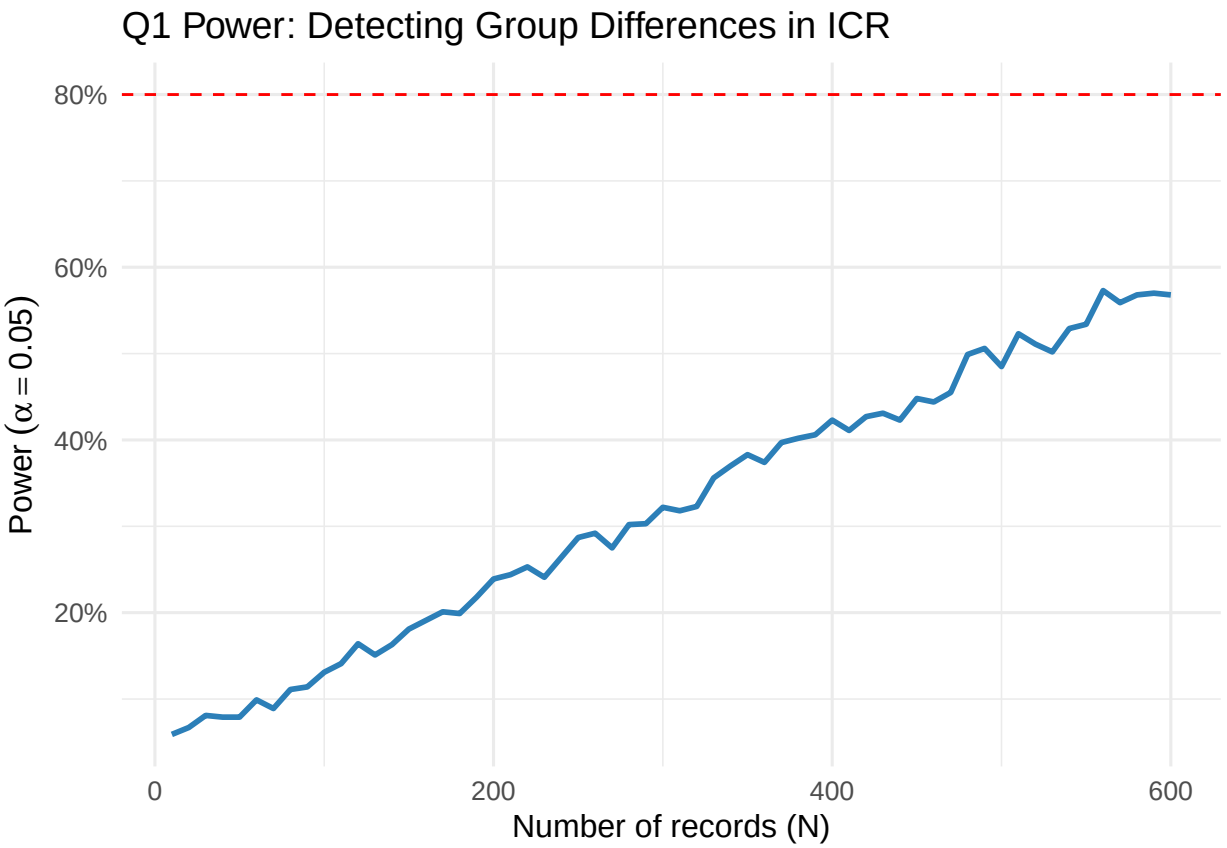
```
      b        = (1 - p_star) * phi,                  # Beta shape parameter beta = (1-mu)·phi
      ICR      = rbeta(1, a, b),                      # Draw a simulated ICR for this record×group
      ICR      = clip01(ICR, 1e-6),                   # Guard against exact 0 or 1
      y        = logit(ICR)                           # Outcome for modeling: logit(ICR)
    ) |>
    ungroup()

  # Fit LMM on logit(ICR)
  fit <- suppressMessages(suppressWarnings(lmer(y ~ condition * group + (1|record), data = out, REML = 
  # ^ Mixed model:
  #   - Fixed effects: condition (AI vs Manual), group (3 levels), and their interaction
  #   - Random intercept: (1|record) to account for record-level correlation across groups
  #   - Outcome: logit-transformed ICR

  # Type III test for group main effect
  an <- anova(fit, type = 3)  # from lmerTest
  p_group <- an["group","Pr(>F)"]  # Extract p-value for the group main effect (our target for Q1 power
  return(p_group)  # Return the p-value for this one simulated dataset
}

# ---- Run simulations across N ----
res <- lapply(N_grid, function(N) {
  pvals <- replicate(reps, simulate_once(N))  # Repeat the simulate-fit-test cycle 'reps' times for eac
  tibble(N = N, power = mean(pvals < alpha, na.rm = TRUE))  # Power = proportion of p-values < alpha
}) |> bind_rows()
# ^ 'res' now contains, for each N, the estimated power to detect a group main effect,
#   given the assumptions above (means, variances, distributional choices, etc.).

# ---- Plot and print minimal N for 80% power ----
p <- ggplot(res, aes(N, power)) +
  geom_line(color = "#2c7fb8", linewidth = 1) +             # Power curve over N
  geom_hline(yintercept = 0.80, linetype = "dashed", color = "red") +  # 80% target line
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +  # Show y-axis as percentages
  labs(title = "Q1 Power: Detecting Group Differences in ICR",
       x = "Number of records (N)",
       y = expression(Power ~ (alpha == 0.05))) +
  theme_minimal(base_size = 12)
print(p)  # Display the power curve
```




```
minN <- res |> filter(power >= 0.80) |> slice_head(n=1)
# ^ Identify the smallest N on our grid whose estimated power meets/exceeds 80%.

if (nrow(minN)==0) {
  message("No N in grid reached 80% power. Increase N or effect size or reps.")
  # ^ If nothing hits 80% within the current grid and assumptions, we suggest increasing N,
  #   the effect size (i.e., widening group differences in mu_manual), or the number of reps
  #   (to reduce Monte Carlo error in the estimate).
} else {
  print(glue::glue("Smallest N with greater than 80% power (Q1): N = {minN$N}, estimated power = {scales
  # ^ Report the smallest N achieving the target and the corresponding estimated power.
}
```

```
## No N in grid reached 80% power. Increase N or effect size or reps.
```

Question 2: Between- AI Group ICR

Does inter-coder reliability vary significantly between with- and without-AI coder groups?

$$Y_{r,g} = \beta_0 + \beta_{\text{AI}} \cdot \mathbf{1}\{\text{Cond}_r = \text{AI}\} + \beta_{\text{Student}} \cdot \mathbf{1}\{g = \text{Student}\} + \beta_{\text{Clinician}} \cdot \mathbf{1}\{g = \text{Clinician}\} \tag{6}$$

$$+ \beta_{\text{AI}\times\text{Student}} \cdot \mathbf{1}\{\text{Cond}_r = \text{AI}\} \cdot \mathbf{1}\{g = \text{Student}\} + \beta_{\text{AI}\times\text{Clinician}} \cdot \mathbf{1}\{\text{Cond}_r = \text{AI}\} \cdot \mathbf{1}\{g = \text{Clinician}\} \tag{7}$$

$$+ u_r + \varepsilon_{r,g}, \tag{8}$$

$$u_r \sim \mathcal{N}(0, \sigma^2_{\text{record}}) \tag{9}$$

$$\varepsilon_{r,g} \sim \mathcal{N}(0, \sigma^2). \tag{10}$$

```r
# Q2 POWER: AI vs Manual effect on ICR (across-records)
# Same data-generating mechanism as Q1, but now delta_AI != 0 and the test is for the condition main ef
# ------------------------------------------------------------------------------------------
# GOAL (plain English):
#   Estimate the statistical power to detect an AI effect on inter-coder reliability (ICR),
#   measured via the Jaccard index, when AI vs Manual is assigned at the RECORD level.
#
# DESIGN IN THIS SIMULATION:
#   * 3 coder groups (Student, Clinician, Coder); 2 coders per group in the study design.
#   * For ICR, the analysis unit is record × group; with 2 coders/group this is a single pairwise Jacca
#   * Each record is assigned to exactly one condition for everyone (AI or Manual).
#   * The model includes a random intercept for record to account for difficulty differences across rec
#
# WHAT "POWER" MEANS HERE:
#   For each N (number of records), we simulate many datasets, fit the model, and record the p-value
#   for the condition (AI vs Manual) main effect. Power = proportion of simulated p-values < alpha.
# ------------------------------------------------------------------------------------------

# Set random seed
set.seed(42)  # Ensures reproducibility: the same random draws each time you run the script

# ---- ASSUMPTIONS TO SET ----
alpha <- 0.05  # Significance threshold for the hypothesis test (Pr(>F) < 0.05 => reject H0)
N_grid <- seq(N_min, N_max, by = steps)  # Sequence of record counts N over which to evaluate power

# Baseline Manual ICR means by group (probability scale)
mu_manual <- c(Student   = 0.58,  # Expected mean ICR for Students when condition = Manual
               Clinician = 0.60,  # Expected mean ICR for Clinicians when condition = Manual
               Coder     = 0.62)  # Expected mean ICR for Coders when condition = Manual
# These encode the group differences you expect BEFORE any AI effect is applied.

# AI lift on ICR (absolute on prob scale) -- SET THIS after literature/decision.
# Consider trying a grid like +0.03, +0.05, +0.10 in separate runs.
delta_AI <- 0.05   # PLACEHOLDER effect size: absolute increase in mean ICR under AI vs Manual
sd_record <- 0.50  # logit-scale SD for record intercept (captures across-record difficulty variability
phi <- 1.5          # Beta concentration (precision); larger phi => less dispersion around the mean ICR
# Notes:
#   * sd_record inflates or reduces differences among records (shared by all groups on that record).
#   * phi controls how tightly simulated ICR values cluster around their mean (on the probability scale

logit  <- function(p) qlogis(p); invlog <- function(x) plogis(x)  # Helper transforms: logit and invers
clip01 <- function(x, eps = 1e-6) pmin(pmax(x, eps), 1 - eps)     # Prevent exact 0 or 1 (avoid infinit

simulate_once <- function(N) {
  # Assign conditions to records in a (near) balanced way so both AI and Manual appear.
  # Each record's condition is shared by all three groups for that record.
  cond <- sample(rep(c("AI", "Manual"), each = N / 2), size = N, replace = FALSE)

  # Draw a record-level random intercept u_rec ~ N(0, sd_record^2) on the LOGIT scale.
  # This represents latent difficulty or other record-specific factors that shift ICR for all groups.
  u_rec <- rnorm(N, 0, sd_record)

  # Build the record × group dataset, then simulate ICR values from a Beta distribution:
```

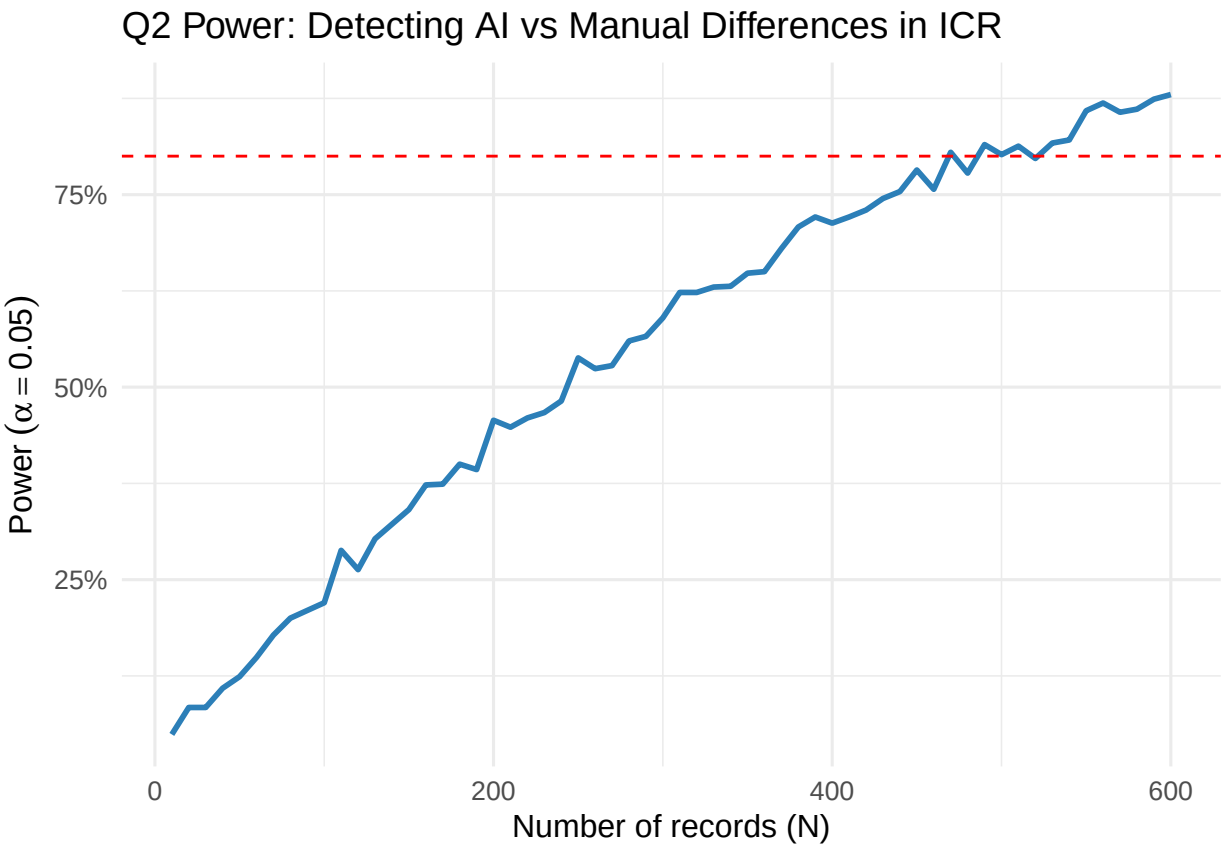
```r
  #   1) Compute the group- and condition-specific mean ICR for each row (probability scale).
  #   2) Shift that mean by the record random effect on the LOGIT scale.
  #   3) Convert back to probability to get the Beta mean (p_star).
  #   4) Use phi to set the Beta's precision (a = p_star*phi, b = (1-p_star)*phi).
  out <- expand.grid(record = seq_len(N),
                     group  = c("Student","Clinician","Coder"),
                     KEEP.OUT.ATTRS = FALSE) |>
    dplyr::mutate(condition = cond[record]) |>
    rowwise() |>
    mutate(
      p_manual = mu_manual[group],                          # Baseline ICR (Manual) for this group
      p_mean   = ifelse(condition=="AI", p_manual + delta_AI, p_manual),  # Apply AI lift if AI
      p_mean   = clip01(p_mean),                            # Keep the mean strictly within (0,1)
      eta      = logit(p_mean) + u_rec[record],             # Add record-level shift on the LOGIT scale
      p_star   = invlog(eta),                               # Back to probability scale after shift
      a        = p_star * phi,                              # Beta shape alpha = mu·phi
      b        = (1 - p_star) * phi,                        # Beta shape beta = (1-mu)·phi
      ICR      = rbeta(1, a, b),                            # Simulated ICR draw for this record×group
      ICR      = clip01(ICR, 1e-6),                         # Guard against exact 0/1 (numerical stabili
      y        = logit(ICR)                                 # Model outcome: logit(ICR)
    ) |>
    ungroup()

  # Fit the linear mixed model (Gaussian LMM) on the logit-transformed ICR:
  #   Fixed: condition + group + condition:group
  #   Random: (1 | record) to capture correlation among group rows for the same record
  fit <- suppressMessages(suppressWarnings(lmer(y ~ condition + group + condition:group + (1|record),
                                data = out, REML = TRUE)))

  # p-value for condition main effect (AI vs Manual), from a Type III ANOVA table
  an <- anova(fit, type = 3)
  p_cond <- an["condition","Pr(>F)"]
  return(p_cond)  # Return the p-value for this simulated dataset
}

# For each N in N_grid, run 'reps' simulated datasets, fit the model, and compute power as
# the proportion of p-values for the condition effect that are < alpha.
res <- lapply(N_grid, function(N) {
  pvals <- replicate(reps, simulate_once(N))              # Vector of p-values across replicates
  tibble(N = N, power = mean(pvals < alpha, na.rm = TRUE))# Estimated power at this N
}) |> bind_rows()

# Plot power vs N with a horizontal line at the 80% target level
p <- ggplot(res, aes(N, power)) +
  geom_line(color = "#2c7fb8", linewidth = 1) +
  geom_hline(yintercept = 0.80, linetype = "dashed", color = "red") +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(title = "Q2 Power: Detecting AI vs Manual Differences in ICR",
       x = "Number of records (N)",
       y = expression(Power ~ (alpha == 0.05))) +
  theme_minimal(base_size = 12)
print(p)  # Display the power curve
```




```
# Identify and report the smallest N on our grid that achieves at least 80% power
minN <- res |> filter(power >= 0.80) |> slice_head(n=1)
if (nrow(minN)==0) {
  message("No N in grid reached 80% power. Increase N or effect size or reps.")
  # If nothing reaches 80%, consider:
  #   * Increasing N (more records),
  #   * Assuming a larger AI effect (delta_AI),
  #   * Increasing 'reps' to reduce Monte Carlo uncertainty in the power estimate.
} else {
  print(glue::glue("Smallest N with greater than 80% power (Q2): N = {minN$N}, estimated power = {scales
  # Reports the minimal N achieving the target and the corresponding estimated power.
}
```

```
## Smallest N with greater than 80% power (Q2): N = 470, estimated power = 80.5%
```

```
# ------------------------------------------------------------------------------------------
# PRACTICAL NOTES (for stakeholders):
# * "Across-records": Because each record is assigned to exactly one condition (AI or Manual),
#   the AI effect is identified by comparing ICR across the set of AI records vs the set of Manual reco
#   while the random intercept (1|record) absorbs some record difficulty variation shared across groups
# * 'phi' (precision) and 'sd_record' (record-level variability) materially influence power:
#   higher variability -> lower power, all else equal.
# * 'delta_AI' is the effect size: try multiple values (e.g., 0.03, 0.05, 0.10) to see how N requiremen
# * This code assumes required packages (lme4, lmerTest, dplyr, ggplot2, tibble, glue) are already atta
# ------------------------------------------------------------------------------------------
```

Question 3: Between-Coder Group Speed

Does coding speed vary significantly between coder groups (student, clinician, coder)?

$$log(time)_{r,i} = \alpha_0 + \alpha_{AI} * Cond_r + \alpha_{Group} * Group_i + \gamma * TrainIndex_{r,i} + \beta_i^{coder} + \beta_r^{record} + \epsilon_{r,i} \quad (11)$$

$$\beta_i^{coder} \sim N(0, \tau_{coder}^2) \quad (12)$$

$$\beta_r^{record} \sim N(0, \tau_{record}^2) \quad (13)$$

$$\epsilon_{r,i} \sim N(0, \sigma^2) \quad (14)$$

```
# Q3 POWER: Group effect on log-time per record
# Model: log(time) ~ Group + Condition + TrialIndex + (1|Coder) + (1|Record)
# Test: Group main effect. Alpha = 0.05. Power vs N.
# ------------------------------------------------------------------------------
# PURPOSE (plain English):
#   Estimate statistical power to detect differences in coding time across
#   coder groups (Student, Clinician, Coder), when modeling log-transformed time
#   with mixed effects for coder and record. Each "power" point is the fraction
#   of simulated datasets (at a given N) where the p-value for the GROUP term
#   is below alpha (0.05).
# ------------------------------------------------------------------------------

# Set random seed
set.seed(42)  # Reproducibility: ensures the same random draws during simulation

# ---- ASSUMPTIONS TO SET ----
alpha <- 0.05  # Significance threshold for testing the group main effect
N_grid <- seq(N_min, N_max, by = steps)  # Range of record counts N to evaluate

coders_per_group <- 2                          # Study design: 2 coders in each group
groups <- c("Student","Clinician","Coder")     # The three coder groups
M <- length(groups) * coders_per_group         # Total coders (not directly used below)

# Baseline geometric mean time (Manual) for Coders (seconds); adjust per domain.
gm_manual_coder <- 120   # 2 minutes per record (placeholder; inpatient times can be longer)
# NOTE: We model times on the *log* scale, so multiplicative effects (e.g., 25% slower)
#       become additive shifts in log-time.

# Group multiplicative effects on time (relative to Coders)
ratio_student   <- 1.25  # students are 25% slower (placeholder)
ratio_clinician <- 1.10  # clinicians are 10% slower (placeholder)
# INTERPRETATION: A ratio > 1 means slower than Coders; e.g., 1.25 = 25% longer times.
# On the log scale this is log(1.25) added to Student rows, etc.

# AI time ratio (overall) -- set to 1.0 here because Q3 focuses on group effect
ai_ratio <- 1.00  # No AI speed-up/slow-down is simulated in this Q3 power analysis

# Learning curve slope on log-scale per additional record (negative => faster over time)
learn_slope <- -0.002  # small per-record improvement (placeholder)
# INTERPRETATION: per additional record attempted (centered within coder),
#                 log-time decreases slightly (i.e., practice effect).

# Random-effect SDs on log-time
```

```r
sd_coder  <- 0.20  # SD of coder-specific random intercepts (heterogeneity among coders)
sd_record <- 0.50  # SD of record-specific random intercepts (difficulty variability across records)
sd_resid  <- 0.20  # Residual SD at observation level (within coder×record noise)
# INTERPRETATION (multiplicative on original time scale): exp(SD) is the typical multiplicative spread.
#   e.g., exp(0.35) = 1.42 implies +/-42% fluctuation around expected time for the same coder & record.

simulate_once <- function(N) {
  # Assign records to conditions at 50/50
  cond_vec <- sample(rep(c("AI", "Manual"), each = N / 2), size = N, replace = FALSE)
  # ^ Each record is assigned to a single condition (AI or Manual), shared by all coders.
  #   Balanced assignment improves numerical stability (both levels present).

  # Build coder roster
  df <- expand.grid(record = seq_len(N),
                    group  = groups,
                    coder  = seq_len(coders_per_group),
                    KEEP.OUT.ATTRS = FALSE) |>
    dplyr::mutate(coder_id = paste(group, coder, sep = "_"),  # Unique coder identifier (e.g., "Student_
                  condition = cond_vec[record]) |>            # Join record-level condition to each row
    arrange(coder_id, record) |>                              # Order rows: within coder over records
    group_by(coder_id) |>
    mutate(trial_idx = row_number(),                          # 1..N attempt index per coder
           trial_idx_c = scale(trial_idx, center = TRUE,      # Center trial index within coder (no sca
                               scale = FALSE)[,1]) |>
    ungroup()

  # Random intercepts
  b_coder  <- rnorm(length(unique(df$coder_id)), 0, sd_coder); names(b_coder) <- unique(df$coder_id)
  b_record <- rnorm(N, 0, sd_record)
  # ^ b_coder: coder-specific baseline speed differences (on log-time).
  #   b_record: record-specific difficulty differences (shared by all coders for that record).

  # Fixed effects construction
  log_gm_coder_manual <- log(gm_manual_coder)  # Baseline (Coders, Manual) on log scale
  group_ratio <- c(Student = ratio_student, Clinician = ratio_clinician, Coder = 1.0)  # Group multipli
  cond_ratio  <- c(AI = ai_ratio, Manual = 1.0)                                        # Condition multi

  df <- df |>
    mutate(
      # 'base' is the expected log-time given fixed effects + random intercepts:
      base = log_gm_coder_manual
             + log(group_ratio[group])            # group effect (multiplicative on time -> additive on
             + log(cond_ratio[condition])         # condition effect (here 0 because ai_ratio = 1.00)
             + learn_slope * trial_idx_c          # small learning (practice) effect per coder
             + b_coder[coder_id] + b_record[record],  # random intercepts for coder and record
      log_time = base + rnorm(n(), 0, sd_resid),  # add residual noise at the observation level
      time_sec = exp(log_time)                    # convert back to seconds (for exclusion rule only)
    ) |>
    # Apply your exclusion (>10 minutes)
    filter(time_sec <= 600)
  # ^ Mirrors the planned QC step: extremely long times are excluded prior to modeling.

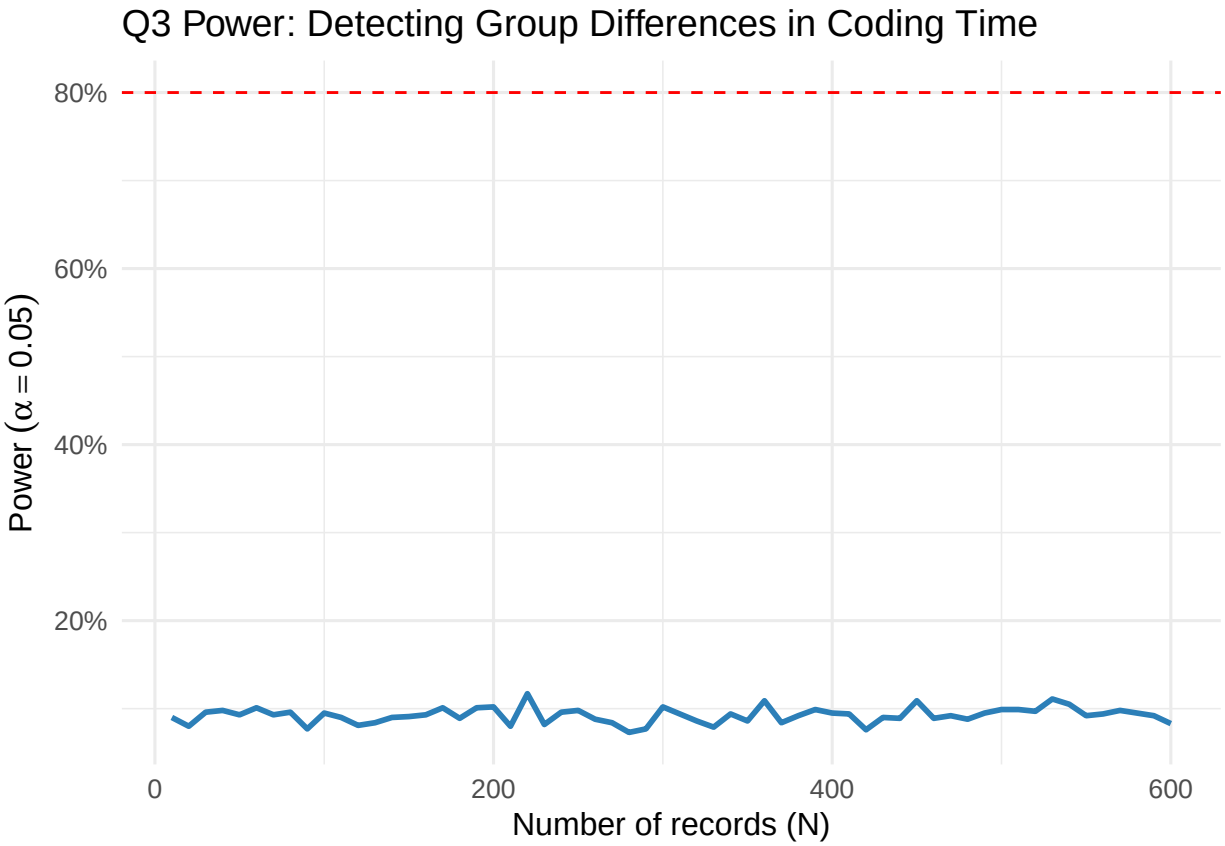
  fit <- suppressMessages(suppressWarnings(lmer(log_time ~ group + condition + trial_idx_c + (1|coder_i
```

```
                                data = df, REML = TRUE)))
  # ^ Mixed-effects model on log-time:
  #   - Fixed effects: group (target of test), condition (balanced but set to no effect here), trial in
  #   - Random intercepts: coder_id (accounts for coder heterogeneity), record (accounts for record dif
  #   - REML: common for variance component estimation in LMMs

  an <- anova(fit, type = 3)  # Type III ANOVA table to test main effects after accounting for others
  p_group <- an["group","Pr(>F)"]  # Extract p-value for the group main effect (our target)
  return(p_group)  # Return the p-value for this simulated dataset
}

# For each N, simulate 'reps' datasets, fit the model, collect p-values for the group effect,
# and compute power as the fraction of p-values below alpha.
res <- lapply(N_grid, function(N) {
  pvals <- replicate(reps, simulate_once(N))             # Vector of p-values across replicates
  tibble(N = N, power = mean(pvals < alpha, na.rm = TRUE))  # Estimated power at this N
}) |> bind_rows()

# Plot the power curve (power vs. N) with a dashed 80% reference line
p <- ggplot(res, aes(N, power)) +
  geom_line(color = "#2c7fb8", linewidth = 1) +
  geom_hline(yintercept = 0.80, linetype = "dashed", color = "red") +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(title = "Q3 Power: Detecting Group Differences in Coding Time",
       x = "Number of records (N)",
       y = expression(Power ~ (alpha == 0.05))) +
  theme_minimal(base_size = 12)
print(p)  # Display the power curve
```




```
# Identify the smallest N achieving at least 80% power on this grid,
# and print a human-readable message with the result.
minN <- res |> filter(power >= 0.80) |> slice_head(n=1)
if (nrow(minN)==0) {
  message("No N in grid reached 80% power. Increase N or group differences.")
  # If 80% isn't reached, options:
  #   - Increase N (more records per coder),
  #   - Assume larger between-group differences (increase ratios),
  #   - Reduce variance assumptions (if justified by data),
  #   - Increase 'reps' for more precise estimates (Monte Carlo error).
} else {
  print(glue::glue("Smallest N with greater than 80% power (Q3): N = {minN$N}, estimated power = {scales
}
```

```
## No N in grid reached 80% power. Increase N or group differences.
```

```
# ------------------------------------------------------------------------
# INTERPRETATION TIPS (for stakeholders):
#   * Because we work on log-time, coefficients translate to PERCENT differences:
#       If beta = log(1.25) = 0.223, that means ~25% longer times.
#   * Random-effects SDs (sd_coder, sd_record, sd_resid) quantify variability at:
#       - Coder level (differences in baseline speed),
#       - Record level (differences in inherent difficulty),
#       - Observation level (trial-to-trial noise for a coder on a record).
#   * Larger variances => lower power, all else equal.
#   * The exclusion at 600s should mirror your real QC; sensitivity analyses can test other thresholds.
# ------------------------------------------------------------------------
```

Question 4: Between- AI Group Speed

Does coding speed vary significantly between with- and without-AI coder groups?

$$log(time)_{r,i} = \alpha_0 + \alpha_{AI} * Cond_r + \alpha_{Group} * Group_i + \gamma * TrainIndex_{r,i} + \beta_i^{coder} + \beta_r^{record} + \epsilon_{r,i} \quad (15)$$

$$\beta_i^{coder} \sim N(0, \tau_{coder}^2) \quad (16)$$

$$\beta_r^{record} \sim N(0, \tau_{record}^2) \quad (17)$$

$$\epsilon_{r,i} \sim N(0, \sigma^2) \quad (18)$$

```
# Q4 POWER: AI vs Manual effect on log-time per record
# Same data generation as Q3, but ai_ratio != 1 and the test targets the condition main effect.
# ------------------------------------------------------------------------------------------
# PURPOSE (plain English):
#   Estimate statistical power to detect a difference in coding time between AI and Manual,
#   when time is modeled on the log scale with mixed effects for coder and record.
#   Power at a given N is the fraction of simulated datasets where the p-value for the
#   CONDITION main effect (AI vs Manual) is below alpha (0.05).
# ------------------------------------------------------------------------------------------

# Set random seed
set.seed(42)  # Reproducibility: ensures the same random draws each time you run this simulation

# ---- ASSUMPTIONS TO SET ----
alpha <- 0.05   # Significance threshold for hypothesis testing (Pr(>F) < 0.05 => detect AI effect)
N_grid <- seq(N_min, N_max, by = steps)  # Range of record counts N over which we evaluate power

coders_per_group <- 2                          # Study design: 2 coders per group
M <- length(groups) * coders_per_group         # Total coders (not directly used below)

gm_manual_coder <- 120      # Baseline geometric mean time (seconds) for Coders under Manual
ratio_student   <- 1.25     # Students take 25% longer than Coders (multiplicative effect)
ratio_clinician <- 1.10     # Clinicians take 10% longer than Coders (multiplicative effect)

# AI time ratio (e.g., 0.8 = 20% faster with AI). Set after we pick a target.
ai_ratio <- 0.80   # PLACEHOLDER informed by several reports (interpretation: 20% faster than Manual)

learn_slope <- -0.002  # Learning curve slope on log-time per additional record (centered). Negative =>
sd_coder  <- 0.20      # Random intercept SD on log-time for coder (between-coder variability)
sd_record <- 0.50      # Random intercept SD on log-time for record (between-record difficulty variabil
sd_resid  <- 0.35      # Residual SD on log-time (within coder×record noise / trial-to-trial variabilit
# NOTE on interpretation of SDs: exp(SD) is the typical multiplicative spread on the time scale.
#   e.g., exp(0.35) = 1.42 => within-observation noise can vary times by +/-42% around the expected val

simulate_once <- function(N) {
  # Assign records to conditions in a balanced way (50/50 AI and Manual).
  # Each record's condition is shared by all coders (record-level condition).
  cond_vec <- sample(rep(c("AI", "Manual"), each = N / 2), size = N, replace = FALSE)

  # Build the coder-by-record grid and add the condition per record to each row
  df <- expand.grid(record = seq_len(N),
                    group  = groups,
                    coder  = seq_len(coders_per_group),
```

```
                    KEEP.OUT.ATTRS = FALSE) |>
    dplyr::mutate(coder_id = paste(group, coder, sep = "_"),  # Unique coder identifier (e.g., "Student_
                  condition = cond_vec[record]) |>            # Join record-level condition to each row
    arrange(coder_id, record) |>
    group_by(coder_id) |>
    mutate(trial_idx = row_number(),                          # Attempt number per coder (1..N)
           trial_idx_c = scale(trial_idx, center = TRUE,      # Center within coder to decouple from i
                               scale = FALSE)[,1]) |>
    ungroup()

  # Random intercepts for coder and record (log-time scale)
  b_coder  <- rnorm(length(unique(df$coder_id)), 0, sd_coder); names(b_coder) <- unique(df$coder_id)
  b_record <- rnorm(N, 0, sd_record)

  # Build fixed-effect components on the log-time scale:
  # - Baseline (Coders, Manual) is log(gm_manual_coder)
  # - Group multipliers: Students and Clinicians are slower than Coders (ratios > 1)
  # - Condition multiplier: AI has ai_ratio (<1 for faster)
  log_gm_coder_manual <- log(gm_manual_coder)
  group_ratio <- c(Student = ratio_student, Clinician = ratio_clinician, Coder = 1.0)
  cond_ratio  <- c(AI = ai_ratio,           Manual = 1.0)

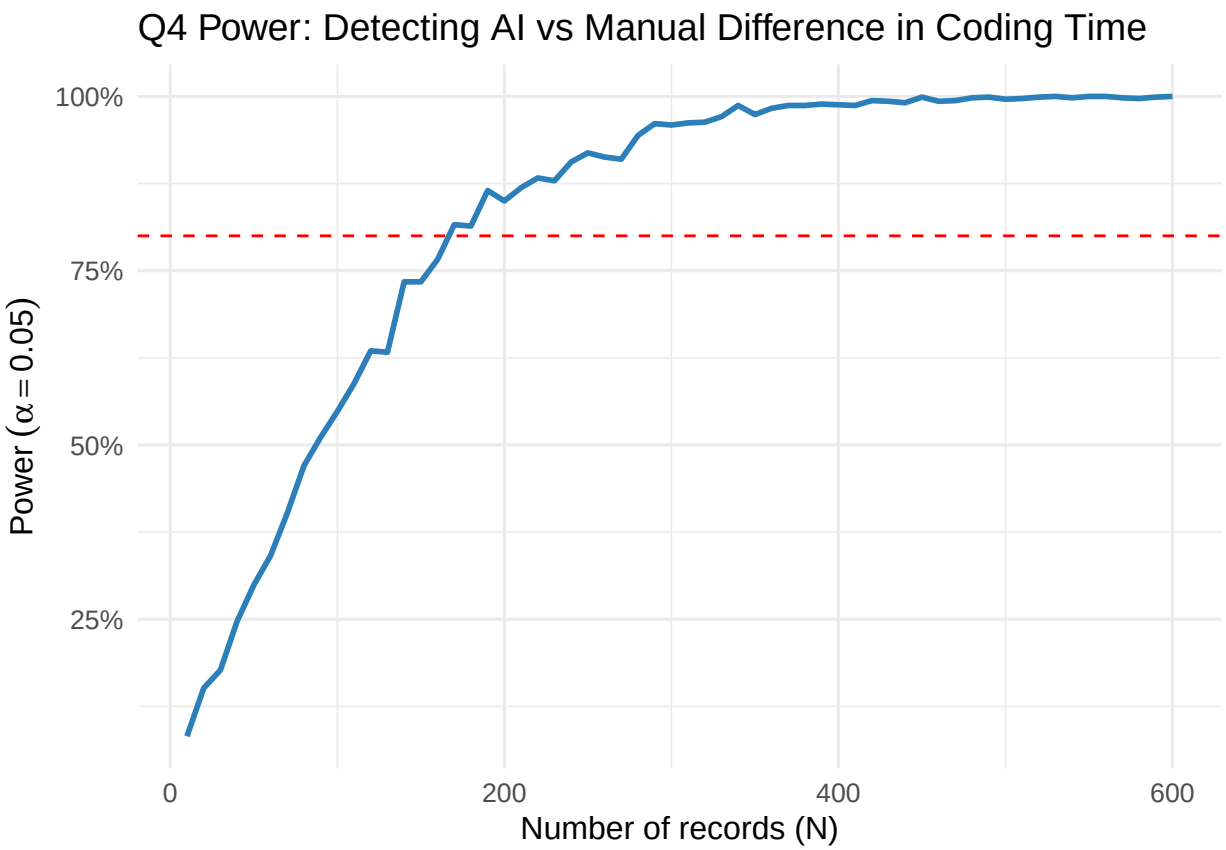
  df <- df |>
    mutate(
      # Expected log-time given fixed effects and random intercepts:
      base = log_gm_coder_manual
             + log(group_ratio[group])       # Group effect (multiplicative on time -> additive on log-
             + log(cond_ratio[condition])    # Condition effect (AI vs Manual; ai_ratio < 1 => faster)
             + learn_slope * trial_idx_c     # Learning effect: small speed-up with experience
             + b_coder[coder_id] + b_record[record],  # Random intercepts: coder- and record-level
      log_time = base + rnorm(n(), 0, sd_resid),  # Add observation-level noise (residual)
      time_sec = exp(log_time)                    # Back-transform for QC filtering only
    ) |>
    filter(time_sec <= 600)  # Apply exclusion: drop times > 10 minutes, mirroring planned QC

  # Fit the linear mixed model on log-time:
  #   Fixed: condition (target), group (nuisance), trial_idx_c (learning)
  #   Random: (1|coder_id) to account for coder heterogeneity,
  #           (1|record)   to account for record difficulty variation.
  fit <- suppressMessages(suppressWarnings(lmer(log_time ~ condition + group + trial_idx_c + (1|coder_id
                               data = df, REML = TRUE)))

  # Extract p-value for the condition main effect (AI vs Manual) from a Type III ANOVA table
  an <- anova(fit, type = 3)
  p_cond <- an["condition","Pr(>F)"]
  return(p_cond)  # Return the p-value for this simulated dataset
}

# For each N in N_grid, run 'reps' simulations; compute power as the share of p-values < alpha
res <- lapply(N_grid, function(N) {
  pvals <- replicate(reps, simulate_once(N))                # Vector of p-values across replicates
  tibble(N = N, power = mean(pvals < alpha, na.rm = TRUE))   # Estimated power at this N
}) |> bind_rows()
```

```r
# Plot the power curve (power vs. N) with a dashed horizontal line at 80%
p <- ggplot(res, aes(N, power)) +
  geom_line(color = "#2c7fb8", linewidth = 1) +
  geom_hline(yintercept = 0.80, linetype = "dashed", color = "red") +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(title = "Q4 Power: Detecting AI vs Manual Difference in Coding Time",
       x = "Number of records (N)",
       y = expression(Power ~ (alpha == 0.05))) +
  theme_minimal(base_size = 12)
print(p)  # Display the power curve
```




```r
# Identify the smallest N on this grid achieving at least 80% power; print a readable message
minN <- res |> filter(power >= 0.80) |> slice_head(n=1)
if (nrow(minN)==0) {
  message("No N in grid reached 80% power. Increase N or AI effect size.")
  # If 80% isn't reached:
  #   - Increase N (more records),
  #   - Assume a stronger AI effect (smaller ai_ratio, e.g., 0.75 = 25% faster),
  #   - Reduce variance assumptions (sd_*), if justified,
  #   - Increase 'reps' for more precise power estimates (less Monte Carlo error).
} else {
  print(glue::glue("Smallest N with greater than 80% power (Q4): N = {minN$N}, estimated power = {scales
}
```

```
## Smallest N with greater than 80% power (Q4): N = 170, estimated power = 81.6%
```

```
# ------------------------------------------------------------------------------------------
# STAKEHOLDER NOTES:
# * Because we model log-time, coefficients translate to percent changes:
#     e.g., ai_ratio = 0.80 implies beta_AI = log(0.80) = -0.223, i.e., ~20% faster with AI.
# * Random effects improve realism:
#     - (1|coder_id): some coders are generally faster/slower.
#     - (1|record): some records are inherently faster/harder to code.
# * The exclusion at 600s reflects your planned QC; consider sensitivity checks (e.g., 900s).
# * Power improves with larger N, stronger AI effects (smaller ai_ratio), or lower variability (sd_code
# ------------------------------------------------------------------------------------------
```